

Experiment No: 2

Implement a program to perform Encryption and Decryption using DES/AES cipher

Name: Rutuja Dundesh Mane

Roll No: A44

Div: A

PRN: 2324000718

ServerAES.java:

```
import java.awt.*;
import java.io.*;
import java.net.*;
import java.util.*;
import javax.crypto.*;
import javax.swing.*;
import java.util.Base64;

public class ServerAES extends JFrame {

    private ServerSocket serverSocket;
    private JTextArea logArea;
    private Map<String, String> qpDatabase;

    public ServerAES() {

        setTitle("Exam Server - AES Encryption");
        setSize(600, 400);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        logArea = new JTextArea();
        logArea.setEditable(false);

        add(new JScrollPane(logArea), BorderLayout.CENTER);

        JButton startBtn =
            new JButton("Start Server (8080)");

        startBtn.addActionListener(
            e -> startServer()
        );

        add(startBtn, BorderLayout.SOUTH);

        qpDatabase = new HashMap<>();

        qpDatabase.put(
```

```

        "10-CS101-1",
        "Q1. What is Java?\nQ2. Explain OOP concepts.\nQ3. Write features of Java."
    );

    qpDatabase.put(
        "11-CS102-2",
        "Q1. Information Security?\nQ2. Explain Caesar Cipher.\nQ3. Write advantages and
disadvantages of DES and AES."
    );

    setVisible(true);
}

private void startServer() {

    new Thread(() -> {

        try {

            serverSocket =
                new ServerSocket(8080);

            log("Server started on port 8080...");

            while (true) {

                Socket client =
                    serverSocket.accept();

                log("Client connected");

                handleClient(client);
            }

        } catch (Exception e) {

            log("Server Error: " + e.getMessage());
        }

    }).start();
}

private void handleClient(Socket client) {

    new Thread(() -> {

        try (
            BufferedReader in =
                new BufferedReader(
                    new InputStreamReader(
                        client.getInputStream())
                )
        ) {
    
```

```

        )
    );

    PrintWriter out =
        new PrintWriter(
            client.getOutputStream(),
            true
        )
    ){

        String cls = in.readLine();
        String courseCode = in.readLine();

        in.readLine();

        String sem = in.readLine();

        String keyDB =
            cls + "-" + courseCode + "-" + sem;

        log("Request: " + keyDB);

        String qp =
            qpDatabase.get(keyDB);

        if (qp == null) {

            out.println("ERROR");
            out.println(
                "Question Paper not found"
            );

            return;
        }

        KeyGenerator keyGen =
            KeyGenerator.getInstance("AES");

        SecretKey secretKey =
            keyGen.generateKey();

        Cipher cipher =
            Cipher.getInstance("AES");

        cipher.init(
            Cipher.ENCRYPT_MODE,
            secretKey
        );

        byte[] encryptedBytes =
            cipher.doFinal(qp.getBytes());
    }

```

```

        String encryptedQP =
            Base64.getEncoder()
                .encodeToString(encryptedBytes);

        String encodedKey =
            Base64.getEncoder()
                .encodeToString(
                    secretKey.getEncoded()
                );

        out.println("OK");
        out.println(encodedKey);
        out.println(encryptedQP);

        log("Encrypted QP sent using AES");

    } catch (Exception e) {

        log("Client Error: " + e.getMessage());
    }

    }).start();
}

private void log(String msg) {
    logArea.append(msg + "\n");
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(ServerAES::new);
}
}

```

ClientAES.java:

```

import java.awt.*;
import java.io.*;
import java.net.*;
import java.nio.file.*;
import javax.crypto.*;
import javax.crypto.spec.SecretKeySpec;
import javax.swing.*;
import java.util.Base64;

public class ClientAES extends JFrame {

    private JTextField classField, courseCodeField, semField, keyField;
    private JTextArea qpArea;
    private String encryptedQP;

```

```

public ClientAES() {
    setTitle("Exam Client - AES Decryption");
    setSize(500, 500);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setLayout(new BorderLayout());

    JPanel inputPanel = new JPanel(new GridLayout(5, 2));

    inputPanel.add(new JLabel("Class:"));
    classField = new JTextField("10");
    inputPanel.add(classField);

    inputPanel.add(new JLabel("Course Code:"));
    courseCodeField = new JTextField("CS101");
    inputPanel.add(courseCodeField);

    inputPanel.add(new JLabel("Semester:"));
    semField = new JTextField("1");
    inputPanel.add(semField);

    inputPanel.add(new JLabel("AES Key:"));
    keyField = new JTextField();
    inputPanel.add(keyField);

    JButton requestBtn = new JButton("Request QP");
    requestBtn.addActionListener(e -> requestQP());
    inputPanel.add(requestBtn);

    JButton decryptBtn = new JButton("Decrypt & Download");
    decryptBtn.addActionListener(e -> decryptQP());
    inputPanel.add(decryptBtn);

    add(inputPanel, BorderLayout.NORTH);

    qpArea = new JTextArea();
    qpArea.setEditable(false);
    add(new JScrollPane(qpArea), BorderLayout.CENTER);

    setVisible(true);
}

private void requestQP() {
    try {
        Socket socket = new Socket("localhost", 8080);
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()))
    } {

        out.println(classField.getText());
        out.println(courseCodeField.getText());
    }
}

```

```

        out.println("Dummy");
        out.println(semField.getText());

        String status = in.readLine();

        if ("ERROR".equals(status)) {
            qpArea.setText(in.readLine());
            return;
        }

        String encodedKey = in.readLine();
        encryptedQP = in.readLine();

        keyField.setText(encodedKey);

        qpArea.setText("Encrypted QP:\n\n" + encryptedQP);

    } catch (Exception e) {
        qpArea.setText("Connection Error: " + e.getMessage());
    }
}

private void decryptQP() {
    try {
        byte[] keyBytes =
            Base64.getDecoder().decode(keyField.getText());

        SecretKey key =
            new SecretKeySpec(keyBytes, "AES");

        Cipher cipher = Cipher.getInstance("AES");

        cipher.init(Cipher.DECRYPT_MODE, key);

        byte[] decryptedBytes = cipher.doFinal(
            Base64.getDecoder().decode(encryptedQP)
        );

        String decrypted = new String(decryptedBytes);

        qpArea.setText("DECRYPTED QP:\n\n" + decrypted);

        Files.write(
            Paths.get("QuestionPaper.txt"),
            decrypted.getBytes()
        );

        JOptionPane.showMessageDialog(
            this,
            "QP Downloaded"
        );
    }
}

```

```

    } catch (Exception e) {
        qpArea.setText(
            "Decryption Error: " + e.getMessage()
        );
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(ClientAES::new);
}
}

```

Output:

